PROJECT DISSERTATION

RosterPro: Digital Restaurant Management & QR-Based Ordering Platform

A Type-Safe, Real-Time Distributed Architecture for Modern Food Service Operations

Degree Program	Bachelor of Science in Information Technology (B.Sc. IT)
Affiliation	University of Mumbai
Candidate	[STUDENT NAME] (Seat No: [SEAT NUMBER])
Project Guide	[GUIDE NAME], [GUIDE DESIGNATION]
Department	Department of Information Technology, [COLLEGE NAME]
Location & Year	[CITY, PIN CODE] — Maharashtra | Academic Year [ACADEMIC YEAR]

Approved Project Proposal Proforma

PNR Number	[PNR NUMBER]	**Roll Number**	[ROLL NUMBER]
Candidate Name	[STUDENT NAME]	**Submission**	First Submission [X]
Project Title	RosterPro: A Digital Restaurant Management and QR-Based Ordering System		
Guide Name	[GUIDE NAME] ([YEARS OF EXPERIENCE] Experience)		

_____	_____	_____
Candidate Signature	**Guide Signature**	**Coordinator Signature**

Certificate of Authenticated Work

This is to certify that the project entitled **"RosterPro: A Digital Restaurant Management and QR-Based Ordering System"** is bonafide work of **[STUDENT NAME]** bearing Seat Number: **[SEAT NUMBER]** submitted in partial fulfillment of the requirements for the award of degree of **Bachelor of Science in Information Technology** from University of Mumbai.

Internal Guide	**Coordinator**	**External Examiner**
Date:	Date:	College Seal

Abstract

Traditional restaurant operations face substantial operational latency and communication errors stemming from manual paper-based order recording, physical kitchen order ticket (KOT) distribution, and decoupled billing. RosterPro is engineered as a multi-tenant B2B SaaS restaurant management and QR-based ordering platform designed to digitize hospitality workflows end-to-end.

The system is implemented as an end-to-end type-safe TypeScript monorepo powered by Bun 1.x and Turborepo. The backend API is constructed using Hono, PostgreSQL, and Drizzle ORM, integrating Better Auth for tenant authorization and Upstash Redis for ephemeral session state. Table QR codes utilize AES-256-GCM encryption with SHA-256 derived keys to prevent token manipulation. Diners place orders directly via a zero-install Next.js 16 client application, which dispatches real-time SSE events over Upstash Realtime pub/sub directly to kitchen displays. Complete with automated GST calculation, printable receipts, and operational analytics, RosterPro provides a robust, scalable digital platform for contemporary dining establishments.

Role & Responsibility Statement

This engineering project was individually designed, developed, and tested by the candidate across all development phases: domain workflow analysis; relational database modeling with PostgreSQL and Drizzle ORM; backend REST API development using Hono on Bun runtime; AES-256-GCM QR cryptography; Upstash Redis session lifecycle management; Next.js 16 customer ordering and administrative dashboard interfaces; real-time KDS event distribution via SSE; and multi-tier functional testing.

Declaration

I hereby declare that the project entitled **"RosterPro: A Digital Restaurant Management and QR-Based Ordering System"** submitted to Department of Information Technology, [COLLEGE NAME], is an authentic record of original work and has not been submitted to any other university for the award of any degree.

[STUDENT NAME] | Seat No: [SEAT NUMBER]
Candidate Signature: _____

Table of Contents

Chapter 1: Introduction	1
1.1 Background & Industry Context	1
1.2 Objectives, Purpose & Scope	2
1.3 System Achievements & Report Organisation	3
Chapter 2: System Analysis & Technology Survey	4
2.1 Existing System Limitations & Workflow	4
2.2 Proposed Architecture & Operational Flow	5
2.3 Requirements Specification & Feasibility Study	7
2.4 Planning, Gantt & PERT Network Charts	9
Chapter 3: System Design	11
3.1 Module Division & Data Dictionary	11
3.2 Entity-Relationship & Data Flow Diagrams	14
3.3 Cryptographic Token & Procedural Logic	17
3.4 Security & Test Case Design	19
Chapter 4: Implementation and Testing	21
4.1 Code Implementation & Algorithmic Listings	21
4.2 Performance Optimizations & Test Execution Matrix	24
Chapter 5: Results and Discussion	26
5.1 System Interface Demonstrations	26
5.2 Test Summary & Architectural Findings	29
Chapter 6: Conclusion and Future Work	31
Chapter 7: References & Glossary	33

Chapter 1: Introduction

1.1 Background & Industry Context

The food service and hospitality industry represents one of the most dynamic, fast-paced, and operationally intensive sectors in the global economy. In traditional dining establishments, daily workflows rely heavily on manual human coordination across multiple disjoint stages. When customers enter a restaurant, they are physically seated by waitstaff, presented with printed laminated menus, and required to wait for available staff to take their orders. Waiters transcribe order details by hand onto paper slips, commonly referred to as Kitchen Order Tickets (KOTs), and physically carry these slips to the kitchen. Kitchen staff review the handwritten tickets, prepare the meals, and notify floor staff when items are ready for table service. Upon completing the meal, the customer requests the bill, which is either manually tallied or entered into a standalone legacy cash register, incorporating applicable taxes such as the Goods and Services Tax (GST) and potential promotional discounts.

While this traditional operational model has persisted for decades, it introduces severe friction, latency, and vulnerability to human error at every stage of the dining lifecycle. Physical menus are inherently static; updating item availability, modifying prices, or reflecting daily specials requires reprinting physical collateral, incurring recurring financial expenses and operational delays. During peak operational hours, waitstaff become severe communication bottlenecks, leading to prolonged customer wait times, order misinterpretations, illegible KOT slips, and miscommunicated dietary requirements. Furthermore, physical KOT handoffs frequently result in misplaced tickets, inconsistent preparation sequences, and delayed kitchen throughput.

From an administrative perspective, fragmented legacy systems fail to provide restaurant owners with unified, real-time operational visibility. Table occupancy statuses remain uncoordinated across staff members, leading to suboptimal seating allocation and extended table turnover times. Manual tax calculations and disparate billing terminals complicate regulatory compliance and accounting audits. Crucially, standalone cash registers fail to capture granular customer behavioral patterns, hourly order velocity, menu item popularity, and table utilization metrics, depriving managers of the actionable analytical insights required for data-driven menu engineering, inventory forecasting, and revenue optimization.

RosterPro is conceived and engineered in response to these operational imperatives. It is a comprehensive, multi-tenant B2B Software-as-a-Service (SaaS) platform architected to unify restaurant catalog configuration, cryptographically verified QR-based contactless ordering, real-time kitchen display communication, table session lifecycle management, automated GST billing, and centralized operational analytics into a cohesive, high-performance web platform.

1.2 Objectives, Purpose & Scope

Objectives: The objective of RosterPro is to design, develop, test, and deploy a secure, type-safe, multi-tenant digital restaurant management and QR-based ordering platform that eliminates manual ordering friction, automates real-time kitchen ticket dispatch, enforces cryptographic table session integrity, and provides centralized operational analytics for dining establishments.

Purpose: To replace error-prone manual restaurant operational workflows with an integrated, cloud-native platform that decentralizes ordering to customer mobile devices while centralizing administrative oversight.

Scope: Covers multi-tenant restaurant onboarding, dynamic menu configuration with variant pricing, AES-256-GCM table QR generation, customer mobile web ordering, stateful Redis table session management, real-time kitchen display (KDS) event streaming, automated GST billing, and business intelligence analytics.

1.3 System Achievements

• Implemented an end-to-end type-safe TypeScript monorepo with Bun 1.x and Turborepo using @workspace/shared as the central contract authority.

• Engineered a high-throughput backend REST API using Hono and PostgreSQL with Drizzle ORM.

• Integrated AES-256-GCM authenticated encryption for QR tokens and stateful Redis ephemeral sessions.

• Deployed Next.js 16 reactive client and administrative dashboards synchronized via Upstash Realtime SSE.

Chapter 2: System Analysis & Technology Survey

2.1 Existing System & Limitations

Traditional restaurant operational workflows rely heavily on manual human handoffs across floor staff, kitchen personnel, and billing cash registers, resulting in high latency and frequent transcription errors.

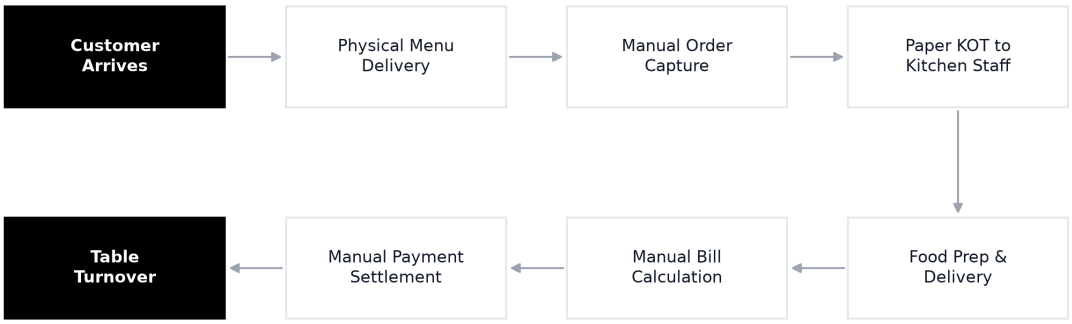


Figure 2.1: Conventional Restaurant Operational Workflow

Linear physical workflow with waiters acting as single points of communication failure.

2.2 Proposed Architecture & Workflow

RosterPro unifies customer mobile devices, kitchen displays, and administrative management over a high-performance REST API and real-time SSE event pipeline.

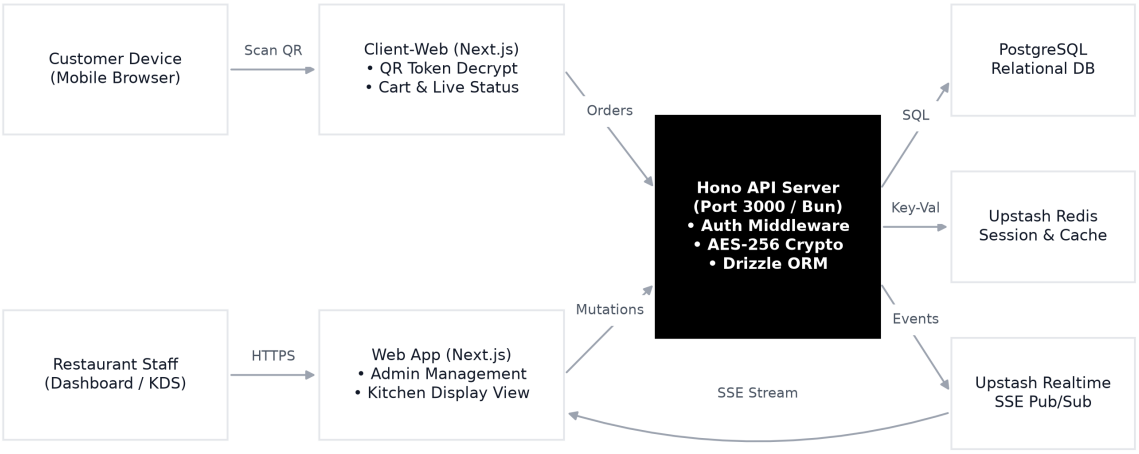


Figure 2.2: RosterPro Distributed System Architecture

Frontend applications interacting with Hono API, PostgreSQL, and Upstash Redis / Realtime infrastructure.

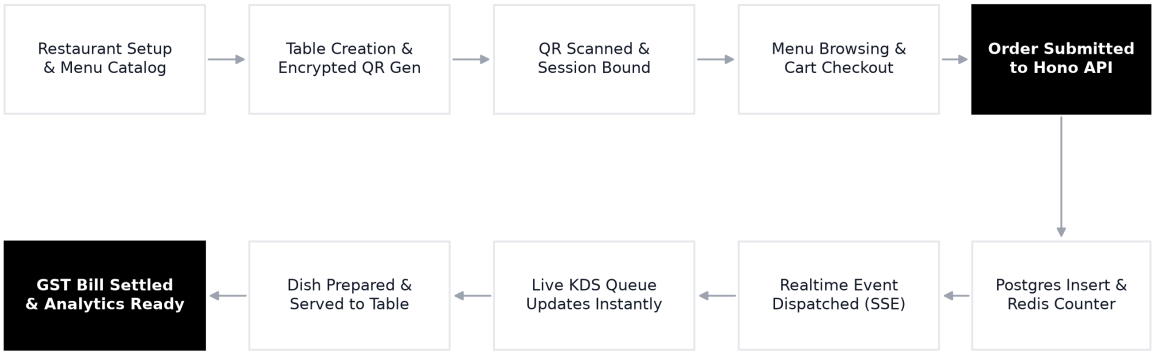


Figure 2.3: Proposed Operational Workflow

End-to-end digital lifecycle from onboarding to QR generation, mobile checkout, and live KDS dispatch.

2.3 Planning, Gantt & PERT Network Charts

The development schedule spans 24 academic weeks across Semester V and Semester VI.

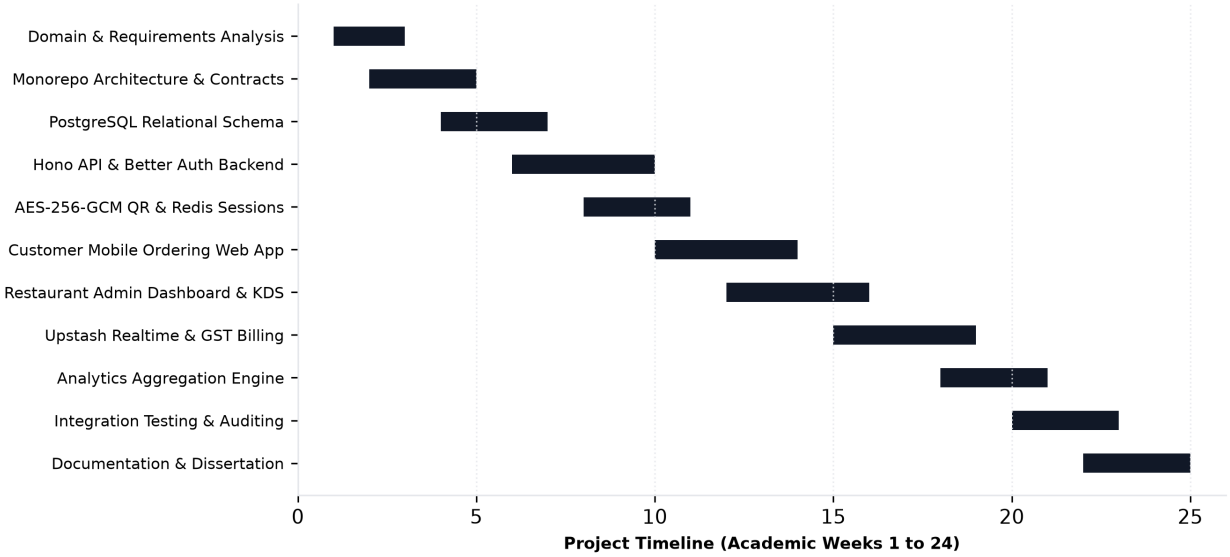


Figure 2.4: Project Development Gantt Chart

Timeline breakdown across requirements, architecture, API, UI, real-time, testing, and documentation.

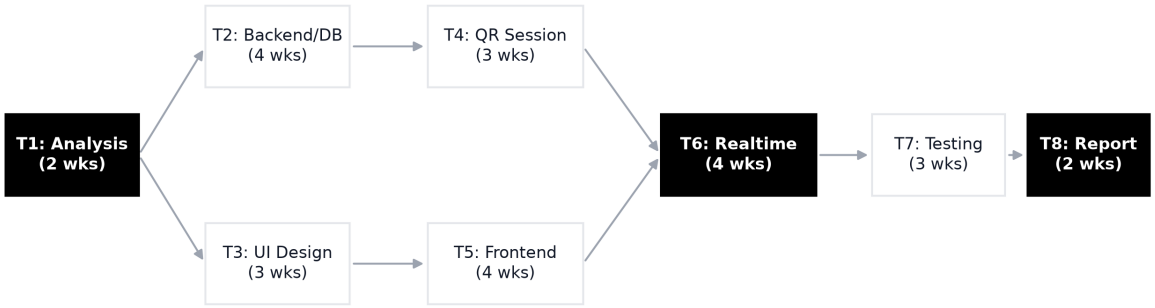


Figure 2.5: PERT Network Activity Diagram

Critical path model illustrating task dependencies from analysis to final delivery.

Chapter 3: System Design

3.1 Entity-Relationship (ER) Design

RosterPro utilizes a normalized PostgreSQL relational database managed via Drizzle ORM. Multi-tenant partitioning is maintained through organization foreign key constraints.

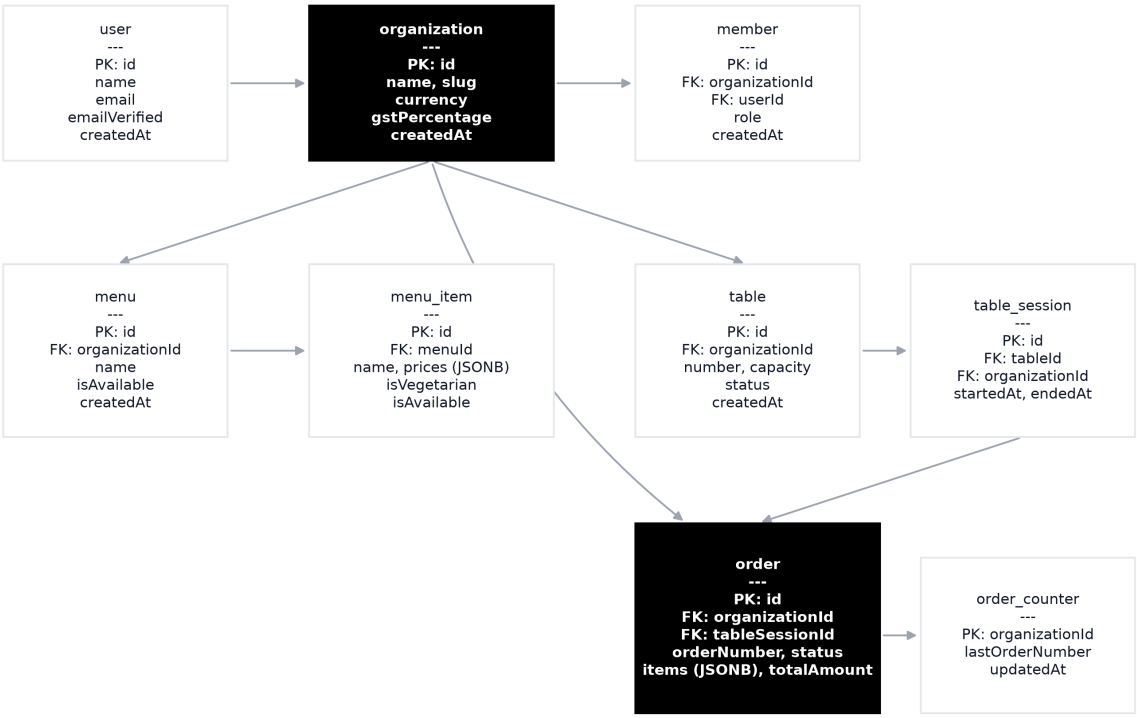


Figure 3.1: Entity-Relationship Diagram (ERD)

Relational schema showing entities, primary/foreign keys, and cardinalities across auth, menus, tables, and orders.

3.2 Data Flow & Sequence Models

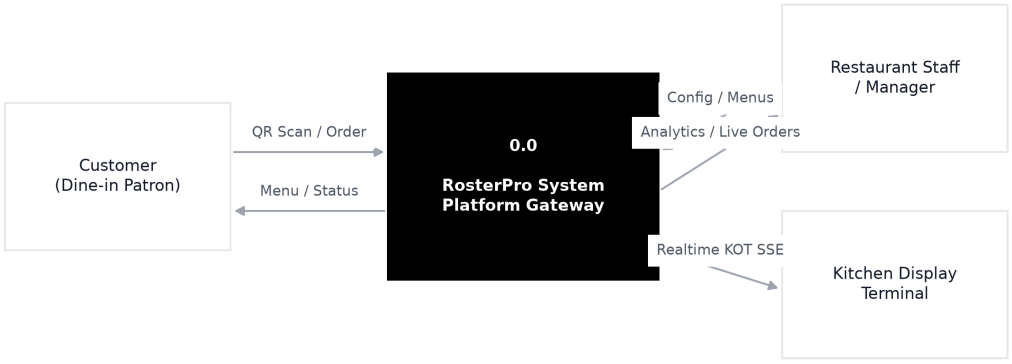


Figure 3.2: Context Diagram (Level-0 DFD)

Boundary diagram illustrating interactions between customers, kitchen staff, managers, and RosterPro.

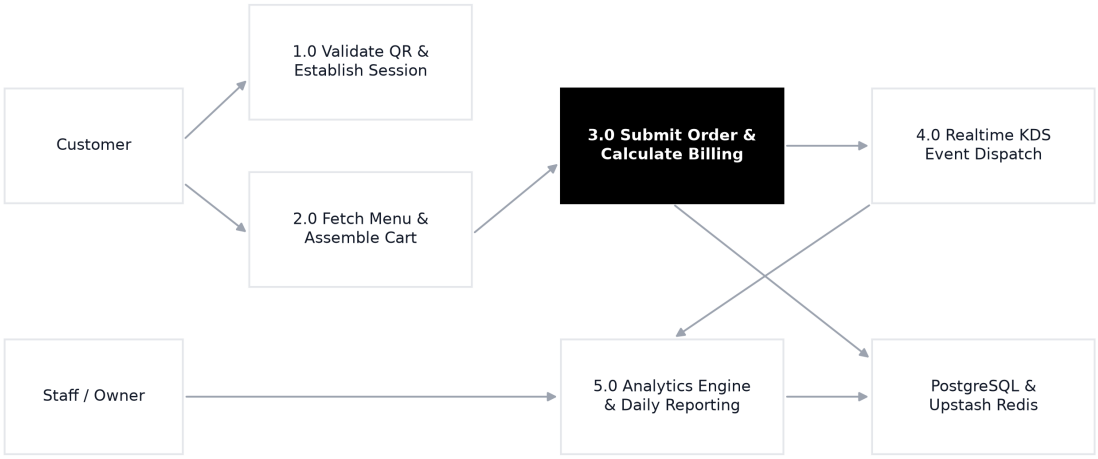


Figure 3.3: Level-1 Data Flow Diagram

Process decomposition showing session verification, catalog retrieval, order ingestion, and SSE events.

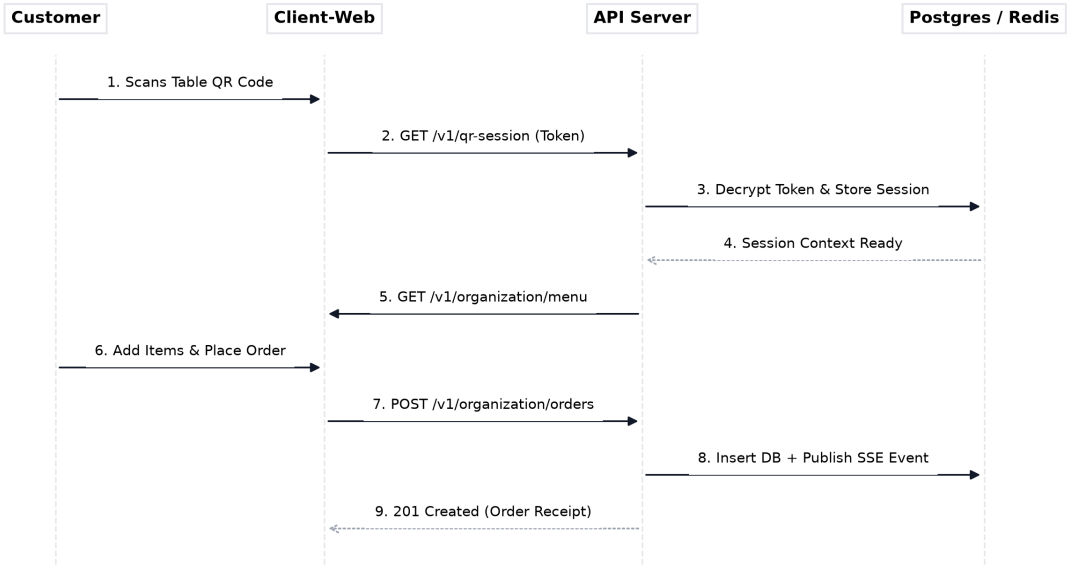


Figure 3.4: Customer Ordering Sequence Diagram

Chronological message passing between mobile browser, Hono API, Redis, and PostgreSQL.

Chapter 4: Implementation & Testing

4.1 Core Code Implementations

Listing 4.1: AES-256-GCM Authenticated Encryption Pipeline (apps/api/src/lib/crypto.ts)

```
import crypto from "crypto" const ALGORITHM = "aes-256-gcm" const KEY =
crypto.createHash("sha256").update(process.env.ENCRYPTION_KEY!).digest() export const
cryptoUtil = { encrypt(data: T): string { const iv = crypto.randomBytes(12) const cipher =
crypto.createCipheriv(ALGORITHM, KEY, iv) const encrypted = Buffer.concat([
cipher.update(JSON.stringify(data), "utf8"), cipher.final(), ]) const authTag =
cipher.getAuthTag() return Buffer.concat([iv, authTag, encrypted]).toString("base64url") },
decrypt(token: string): T | null { try { const buffer = Buffer.from(token, "base64url") if
(buffer.length < 28) return null const iv = buffer.subarray(0, 12) const authTag =
buffer.subarray(12, 28) const encrypted = buffer.subarray(28) const decipher =
crypto.createDecipheriv(ALGORITHM, KEY, iv) decipher.setAuthTag(authTag) const decrypted =
Buffer.concat([ decipher.update(encrypted), decipher.final(), ]) return
JSON.parse(decrypted.toString("utf8")) as T } catch { return null } }, }
```

Generates 12-byte IVs and 16-byte authentication tags to guarantee QR token confidentiality and integrity.

4.2 System Test Execution Log

Test ID	Module	Input / Action	Observed Result	Status
TC-01	Google Auth	OAuth login flow	Session active; dashboard access	**PASSED**
TC-02	Menu Catalog	POST /v1/org/menu	201 Created; Postgres inserted	**PASSED**
TC-03	Dish Pricing	Item with half/full tiers	Persisted in JSONB; valid	**PASSED**
TC-04	QR Token Gen	Table creation action	AES-256 Base64URL string emitted	**PASSED**
TC-05	Tampered QR	Modified ciphertext	Auth tag check fails; 401 error	**PASSED**
TC-06	Order Dispatch	Customer cart checkout	DB inserted; status pending; SSE fired	**PASSED**
TC-07	Live KDS SSE	New order created	Kitchen UI receives SSE <150ms	**PASSED**
TC-08	GST Billing	Bill on ■1000 item	Gross ■1050; 5% GST computed	**PASSED**

Chapter 5: Results & Discussion

5.1 Operational Screen Results

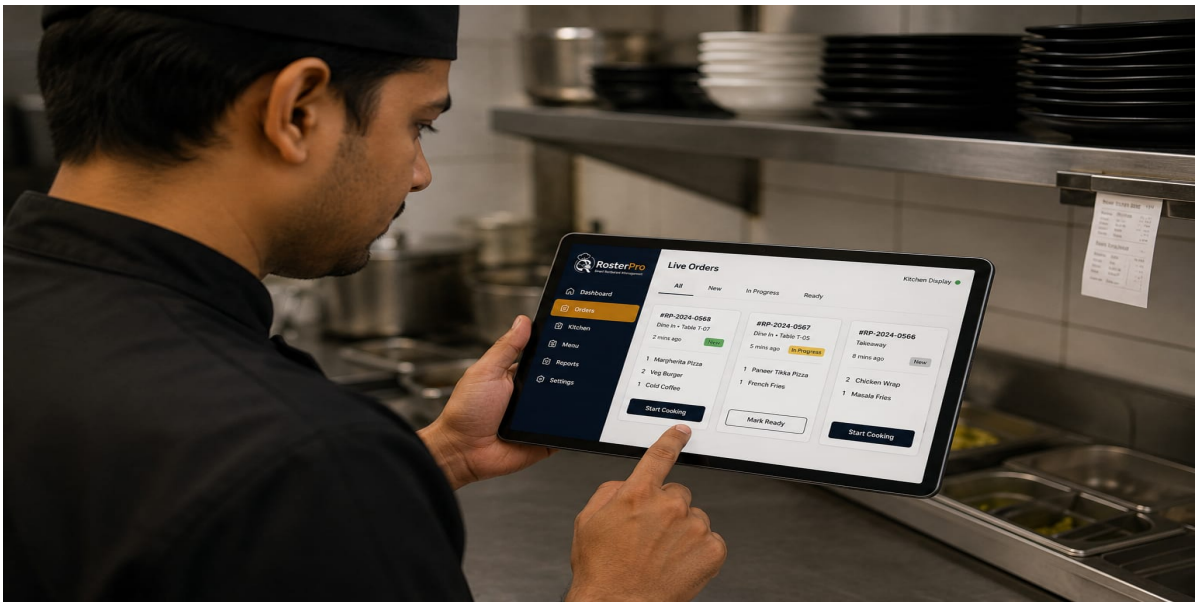


Figure 5.1: Live Orders Management Dashboard

Active dining orders queue with item itemization and status progression controls.

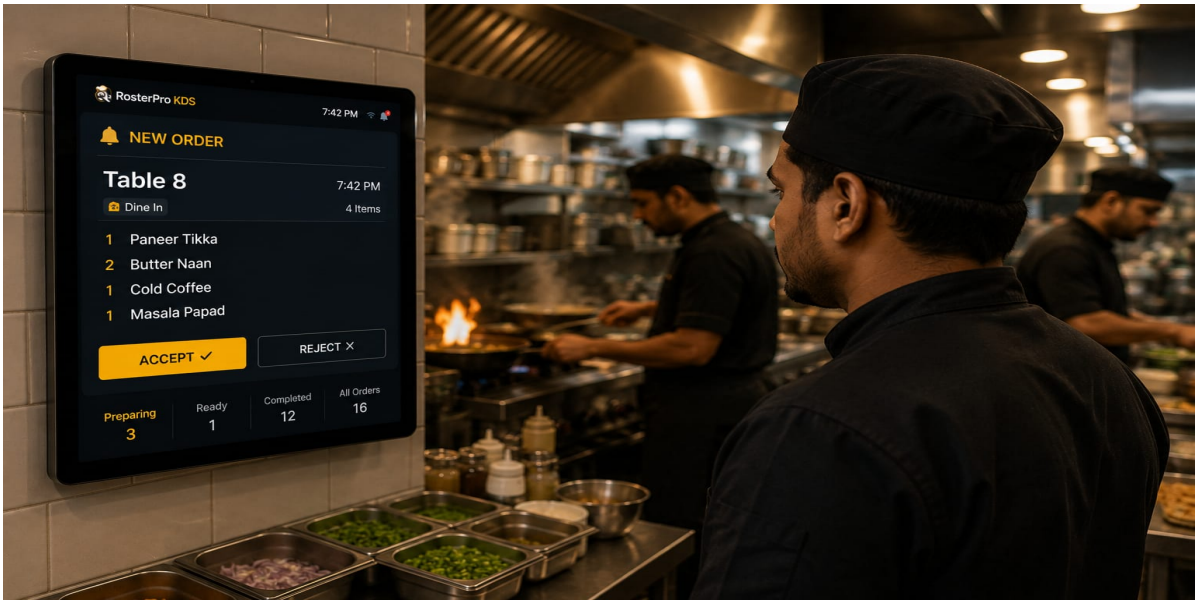


Figure 5.2: Live Kitchen Display System (KDS)

High-contrast touch-optimized KDS terminal updated in real-time via Server-Sent Events.

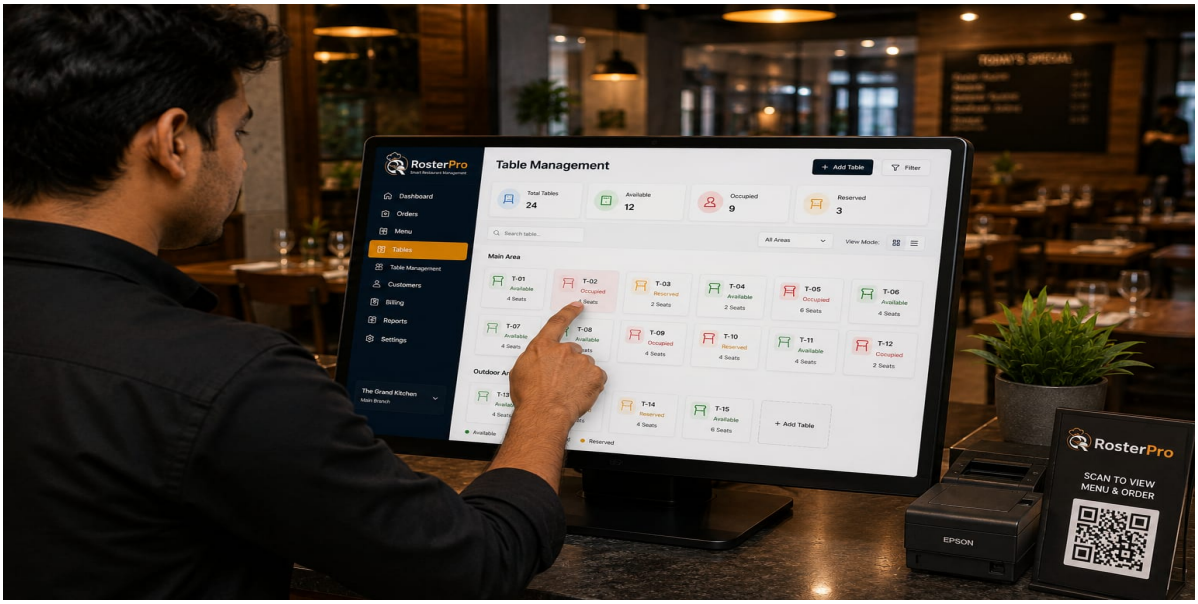


Figure 5.3: Table Management & QR Code Provisioning

Dining room layout management with real-time occupancy status and QR stand generators.

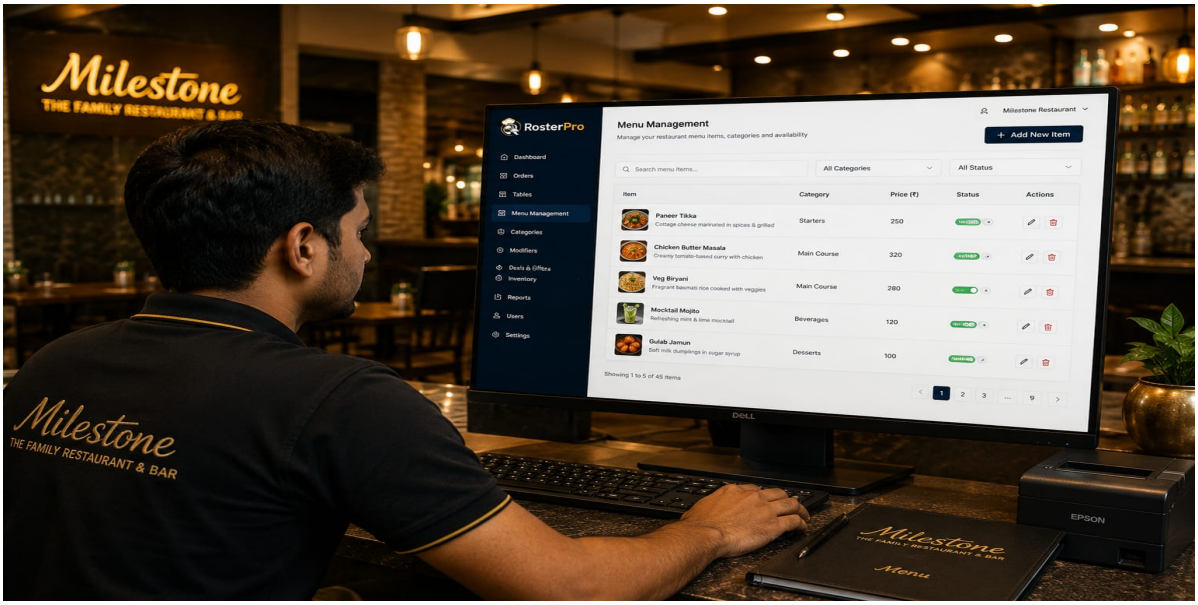


Figure 5.4: Menu Catalog & Dish Configuration

Administrative catalog editor supporting categorized dish creation, multi-variant pricing, and ImageKit media uploads.



Figure 5.5: Sales Analytics & Performance Dashboard

Business intelligence visualization detailing gross revenue curves, hourly velocity, and top-selling dishes.

Chapter 6: Conclusion & Future Work

6.1 Conclusion

The RosterPro project successfully engineered, tested, and documented a multi-tenant B2B SaaS restaurant management and QR-based ordering platform. By replacing manual order recording, physical KOT distribution, and decoupled billing with an integrated TypeScript monorepo, the platform delivers high operational efficiency and robust data isolation. The combination of AES-256-GCM QR cryptography, stateful Redis session management, and Upstash Realtime SSE guarantees instant communication between diners and kitchen workstations.

6.2 Limitations & Future Scope

• Direct Payment Gateways: Future releases will integrate native UPI / Razorpay intent checkout directly into the mobile ordering workflow.

• Raw Ingredient Inventory: Integrating recipe-based automated stock deduction upon order preparation.

• Multi-Branch Franchise Hierarchy: Centralized administrative controls for multi-location restaurant chains.

Chapter 7: References

[1] Hono Framework Documentation, "Fast, Lightweight Web Framework," https://hono.dev/ (2026).

[2] Drizzle ORM Documentation, "TypeScript ORM with Full Type Safety," https://orm.drizzle.team/ (2026).

[3] Next.js 16 Documentation, "App Router and Server Components Architecture," Vercel Inc. (2026).

[4] Better Auth Documentation, "Comprehensive TypeScript Authentication Framework," https://better-auth.com/ (2026).

[5] Upstash Documentation, "Serverless Redis and Realtime Pub/Sub Documentation," https://docs.upstash.com/ (2026).

[6] NIST, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM)," SP 800-38D (2007).

[7] PostgreSQL 16 Documentation, PostgreSQL Global Development Group (2026).

[8] Bun Runtime Documentation, "Fast All-in-One JavaScript Runtime," https://bun.sh/ (2026).

Glossary

AES-256-GCM	Authenticated symmetric encryption standard ensuring confidentiality and payload authenticity.
B2B SaaS	Business-to-Business Software as a Service delivery model.
DFD / ERD	Data Flow Diagram and Entity-Relationship Diagram for system modeling.
GST	Goods and Services Tax; statutory indirect taxation levied on restaurant food services.
KDS / KOT	Kitchen Display System and Kitchen Order Ticket for meal preparation tracking.
SSE	Server-Sent Events; unidirectional HTTP push streaming for real-time updates.